

starRoboHarness: Reliable Evaluation and Efficiency Analysis for Policy-Reasoner Robot Control

starRoboHarness contributors

github.com/starVLA/StarHarness

Abstract Evaluating a robot policy supervised by a multimodal reasoner requires more than a success count. Action cadence, case identity, execution failures, and infrastructure retries can all alter the reported outcome. We present starRoboHarness, a model-neutral control and evaluation harness that binds proposals to observations, checks decisions before robot actions, and records native terminal receipts in an append-only trace. We analyze two historical RoboDojo panels, each containing five tasks and five exact policy-hybrid case pairs per task. In the original panel, the QwenPI_v3 policy supervised by GPT-6 Astra succeeds on 6/25 cases compared with 1/25 for the policy alone. Mean native scores are 43.6 and 9.0. In the added panel, success is 11/25 versus 8/25 and mean score is 50.4 versus 38.4. The effect varies by task. Packing score falls from 20 to 2 despite no successes for either method, while several other tasks gain completions. In the added panel, the hybrid records 462.5 million reported input tokens and 4,627 correction steps, making efficiency a central open problem. These results describe historical matched cases, not a causal estimate of reasoner corrections. We provide case-level evidence and reproducible table generation, then identify measurements needed for a controlled follow-up study.

1. Introduction

Learned robot policies can propose action sequences from images and proprioception. Multimodal reasoners can review those proposals against a task goal, assess visible progress, and request local corrections. Combining them raises two related problems. First, an apparently better outcome may reflect a change in action cadence, reset identity, prompt, or retry policy rather than the reasoner's decisions. Second, an external service failure can be mistaken for a robot failure unless the evaluation records how each episode ended.

We develop starRoboHarness to make the boundary between proposal, decision, and physical execution explicit. The harness binds each proposal to an observation, checks decision freshness and action limits, assigns execution to one controller, and records acknowledgements and native terminal outcomes. Policy, reasoner, and environment implementations live behind adapters. QwenPI_v3, GPT-6 Astra, and RoboDojo form the reference integration studied here. Model weights and simulator assets remain external to the public package.

This paper reconstructs two historical paired panels rather than presenting a prospectively randomized study. Each panel contains five tasks with five exact case pairs per task. We report native success, partial score, paired outcomes, and reasoning usage, then examine a small development validation and an early direct-control case separately. The original panel has 6/25 hybrid successes and 1/25 policy successes. The added panel has 11/25 and 8/25. The gains vary by task, and packing loses partial score under hybrid control. These observations motivate the main question of this paper: which decisions can a reasoner improve, and at what execution and inference cost?

Our contributions are a documented execution contract, a validity-aware reconstruction of 100 terminal method rows, and an analysis of task-level gains and failures. We release sanitized case rows and

code that regenerates the tables from source exports with digest checks. This paper also identifies the controlled measurements needed before attributing a quality gain to a specific harness component.

2. Related Work

StarVLA supplies a modular backbone and action-head codebase for vision-language-action research [1]. Its QwenPI_v3 workstream supplies the policy implementation used here. starRoboHarness supplies the execution and evidence layer around a policy. GPT-as-Policy studies a multimodal reasoner acting directly and reviewing a specialist policy on a ten-task RoboDojo subset with five cases per task [2]. Agent as Policy places a general-purpose agent in direct control of robot manipulation through a robot interface [3]. These studies motivate distinct roles for a learned policy, a direct-control reasoner, and a supervising reasoner. Our contribution is the harness boundary and an analysis of our own terminal receipts. We do not transfer success numbers between protocols.

RoboDojo provides a broader sim-and-real evaluation framework with 42 simulation and 18 real-world tasks [4]. The panels reported here are subsets of simulation tasks with five selected cases per task. They are not a full RoboDojo benchmark run and cannot be compared directly with full-protocol leaderboard values.

3. System Architecture

Figure 1 summarizes the system architecture from task and world state through method selection, execution, adapters, and audit. The task specification, visual observations, robot state, and constraints are distinct inputs. The evaluated method profiles are policy-only execution, a reasoner supervising policy proposals, and direct reasoner control. All three pass through the shared execution and evidence boundary before reaching an environment adapter. Monitoring and native evaluation read the resulting trace.

Demonstrations, task references, and interaction history are possible context sources in the broader design. These belong to a proposed experience-reuse path. Its adaptive execution router and authority arbiter are research targets rather than a component evaluated across the panels in this paper. The same distinction applies to general tool-augmented control, alternative VLA or world-action-model policies, broader safety and risk analytics, human-in-the-loop self-improvement, and deployment to arbitrary physical robots. The reported experiment exercises the QwenPI_v3, Astra, and RoboDojo integration. This status distinction matters because the framework is broader than the evidence in Section 5.

3.1. Execution contract

Each proposal is bound to an exact observation and carries policy and action-contract identity. The adapter checks shape, freshness, bounds, units, gripper semantics, and request identity. A reasoner reviews the previous execution and next proposal using visible images, proprioception, and public trace evidence. Hidden reward, privileged object state, and future simulator truth are outside that interface. One controller owns state-changing execution and records acknowledgements. The native RoboDojo terminal outcome determines success and score. An attempt with uncertain acknowledgement remains in the ledger and is not silently replayed under the same identity.

QwenPI_v3 uses three RGB views, 14-dimensional dual-arm proprioception, and a 50-by-14 absolute-joint proposal. The serving layer performs the pinned `arx_x5` normalization. The native pure-policy cadence executes 16 actions per replan. The historical hybrid uses bounded student prefixes and at

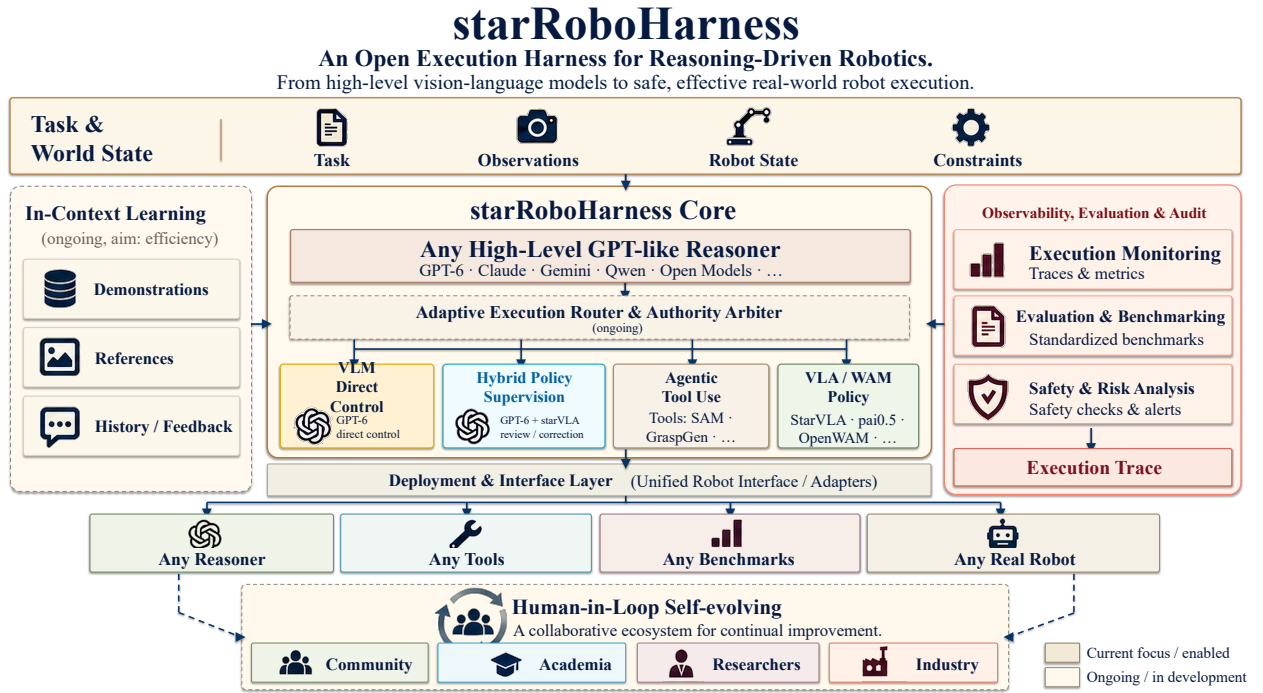


Figure 1 | Project framework. Task information enters the reasoner and execution layer, which connects control modes to robot and benchmark adapters. Monitoring and native outcomes are recorded in the evaluation trace. Components marked ongoing in the diagram are research directions. The reported results use the QwenPI_v3, Astra, and RoboDojo integration.

most five-step local EEF corrections.

4. Experimental Design

4.1. Methods and case selection

The two paired panels compare a pure QwenPI_v3 policy with QwenPI_v3 proposals supervised by GPT-6 Astra. The pure policy executes 16 actions before replanning. The historical hybrid can accept bounded student prefixes or issue local end-effector corrections of at most five steps. Consequently, the comparison includes the supervision strategy and its different execution cadence. It is not an ablation in which the reasoner is the only changed variable. Direct Astra control appears only in the single early development case reported in Appendix B.

The original panel has five tasks and 25 selected cases. The added panel has another five tasks and 25 selected cases. Task names and per-method results appear in Tables 1 and 2. Within each panel, matching uses task, variant, layout identity, seed, and case identifier. The first panel was reconstructed from 50 valid method rows in the formal-five-v58 consolidation. The added panel pairs 25 frozen hybrid cases with 25 later pure-policy receipts. Four valid policy receipts were reused and 21 were newly collected. Packing includes three standard and two random layouts selected identically for both methods.

4.2. Outcome definitions and evidence

The unit of analysis is a native terminal episode. A row enters success-rate and score denominators only when `complete=true`, `valid_for_success_rate=true`, and a simulator terminal receipt exists. Timeouts at the native step limit are valid failures with their native partial score. Infrastructure or provider failures, pending cases, and uncertain control outcomes are excluded and retained in a separate attempt ledger. The bundled generator verifies 50 distinct exact case pairs and checks the digests of its four source exports before writing tables.

For a method m and a task containing $N = 5$ valid cases, success rate is $N^{-1} \sum_i s_{i,m}$, where $s_{i,m}$ is the native binary outcome. Mean score is $N^{-1} \sum_i r_{i,m}$, where $r_{i,m}$ is the native score on a 0–100 scale. We also count cases in which only one method succeeds and cases in which native score increases, decreases, or ties. These paired counts are descriptive. Eighteen pre-control infrastructure attempts in the added-panel baseline remain in the attempt ledger and contribute to neither method’s denominator.

4.3. Historical comparability

The panels were collected in different periods. Appendix C records the source-campaign lineage. Hardware, provider sessions, recovery history, and method cadence were not held constant across all runs. We therefore report two separate paired aggregates. The union of 50 hybrid cases is a descriptive index, not one prospective ten-task randomized trial. Wall-time and token measurements are operational telemetry. They do not by themselves establish a controlled latency or billing comparison.

5. Results

5.1. Original five-task paired panel

Task	Policy succ.	Hybrid succ.	Policy score	Hybrid score
Organize table	0/5	0/5	10.0	65.0
Classify by language	0/5	0/5	0.0	0.0
Imitate sorting	0/5	1/5	1.0	36.0
Arrange largest number	0/5	1/5	2.0	37.0
Fold clothes	1/5	4/5	32.0	80.0
Overall (25)	1/25	6/25	9.0	43.6

Table 1 | Original five-task paired panel. Each task has five exact case pairs. Success is shown as completed cases over valid cases. Score is the mean native score on a 0–100 scale.

Hybrid success is 6/25 (24%) versus 1/25 (4%) for the pure policy. Native mean score is 43.6 versus 9.0 (Table 1). Across exact binary pairs there are five hybrid-only wins, no policy-only wins, and 20 ties. Native score improves in 16 pairs, declines in one, and ties in eight. Fold clothes contributes three of the five additional successes. Its success count rises from one to four and its mean score rises from 32 to 80. Language classification is unsuccessful for both methods and has zero mean score in both groups. The aggregate gain therefore combines a strong task-specific improvement with a task showing no measurable progress. Different student execution cadences and stochastic visual observations prevent attributing the paired difference solely to corrections.

5.2. Added five-task paired panel

Task	Policy succ.	Hybrid succ.	Policy score	Hybrid score
Put bottles in bin	3/5	4/5	64.0	82.0
Classify objects	0/5	0/5	6.0	22.0
Build tower	4/5	4/5	82.0	86.0
Make Kong	1/5	3/5	20.0	60.0
Pack objects in box	0/5	0/5	20.0	2.0
Overall (25)	8/25	11/25	38.4	50.4

Table 2 | Added five-task paired panel. Each task has five exact case pairs. Pure-policy counterparts were collected after the frozen hybrid cases. Success is shown as completed cases over valid cases.

The added panel gives 11/25 (44%) hybrid and 8/25 (32%) pure-policy successes, with mean native scores 50.4 and 38.4 (Table 2). The 25 binary pairs contain four hybrid-only wins, one policy-only win, and 20 ties. Score changes are more mixed: seven positive, three negative, and 15 tied pairs. Make Kong accounts for two extra successes, while bottles accounts for one. Build tower has 4/5 successes under both methods. The added-panel gain is concentrated in a few tasks.

Packing is the largest regression. Both methods fail all five cases, but mean native partial score falls from 20 to 2. The five hybrid cases terminate at the native 1,300-step limit with scores 0, 0, 0, 0, and 10. They are valid scored failures. A recorded live trajectory in this campaign verified a toy-car grasp, then used repeated five-step end-effector transfer corrections. The same trajectory accepted four consecutive 15-step student segments before a later open-gripper hover triggered a bounded takeover. This trace shows delegation and intervention within one episode. It suggests that transfer efficiency and takeover timing deserve closer study. One trace does not establish why all five packing cases failed. Classification also remains at 0/5 success for both methods despite a higher hybrid partial score.

Figure 2 makes the variation visible. Eight tasks have a higher hybrid mean score, one is tied, and packing is lower. A positive partial-score change need not imply task completion. In added-panel classification, score rises from 6 to 22 while success remains 0/5 under both methods. Build tower retains 4/5 successes with a four-point score increase. Success and partial score answer different questions and should be read together.

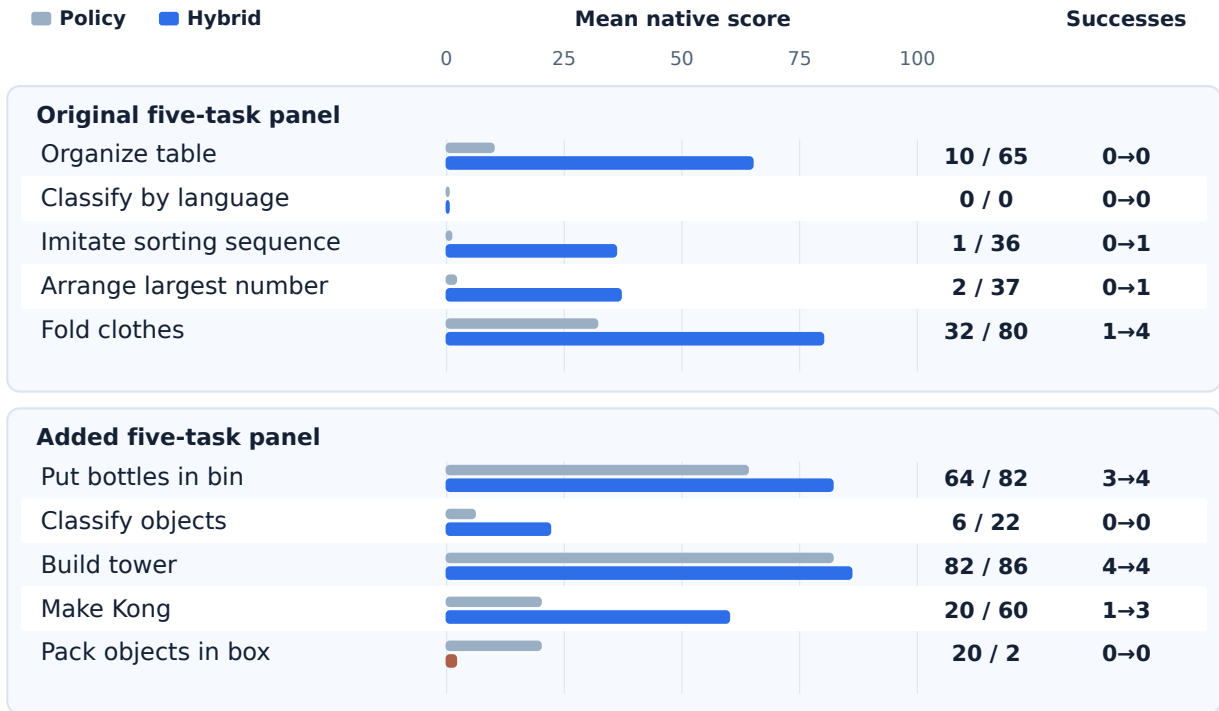


Figure 2 | Task-level mean native score and success count on five matched cases per task. Gray bars show pure policy, blue bars show hybrid, and the rust-colored bar marks the packing regression. Numbers at right give policy/hybrid mean score and policy-to-hybrid success counts. The original and added panels were collected in different periods.

Ten-task hybrid index. The original and added hybrid blocks together contain 17/50 native successes (34%) and mean score 47.0. This is a descriptive union of two historical blocks. Later added-block policy receipts enable the second paired table. They do not turn the ten-task set into one prospectively frozen comparison, so the two paired aggregates remain separate.

5.3. Inference cost and efficiency

Metric, added 25-case panel	Pure policy	Hybrid
Mean control-loop wall time (s)	330.5	4,999.8
Mean policy calls / episode	54.4	85.6
Total correction steps	0	4,627
Total input tokens	0	462,535,681
Cached input fraction	n/a	93.95%
Total output tokens	0	1,563,270

Table 3 | Runtime telemetry on exact added-panel cases. Token counts are the last recorded asynchronous usage snapshots, not billing. Hardware, transport, and collection period differ, so wall-time values are descriptive rather than a controlled slowdown estimate.

Table 3 shows the scale of reasoning used by the added-panel hybrid. Input-token caching is high, yet 462.5 million reported input tokens and 4,627 correction steps were recorded across 25 cases. The roughly 15-fold difference between displayed mean control-loop times is descriptive because

historical machines and transports differ. More reasoning did not guarantee a better outcome in packing. A useful next comparison should measure native quality and cost together on matched hardware, provider routes, and cases.

The development validation and the early direct-control case are reported separately in Appendix B. They are excluded from both paired panels.

6. Discussion

6.1. Interpretation of paired outcomes

Across the two panels, the binary pairs contain nine hybrid-only successes, one policy-only success, and 40 ties. Native score rises in 23 pairs, falls in four, and ties in 23. These pooled counts describe the observed cases, while the original and added panels retain separate collection histories. They show why reporting only overall success would hide useful distinctions. Added-panel classification receives more partial credit without a completed task. Packing loses partial credit even though both methods still time out. Fold clothes and Make Kong contribute several actual completions.

The evidence also separates two possible roles for the reasoner. It can approve student prefixes, and it can take over with bounded local corrections. The archived successful build-tower development case records zero correction steps. The two successful dev cases also record zero corrections. A packing trace contains both accepted student segments and a later takeover, yet its native outcome is failure. None of these observations identifies correction frequency as a general cause of success. They motivate measuring when a reasoner delegates, what visual evidence triggers intervention, and whether an intervention improves the next verified state.

Validity accounting changes the interpretation of failures. The five packing timeouts and the failed classification cases remain in the denominator because RoboDojo issued valid terminal receipts. The 18 added-panel attempts that failed before control remain in the attempt ledger and do not enter the denominator. This distinction prevents an infrastructure retry from becoming either a scored failure or an unreported replacement for a difficult case.

6.2. Limitations and next experiments

Each paired task has only five cases. Historical nodes, provider sessions, recovery attempts, and policy cadence differ. The project framework includes components still under development. Figure 1 marks these as proposed extensions. Section 3.1 specifies the evaluated control path. Published GPT-as-Policy and StarVLA results use different task suites and episode counts. We report public decision traces, acknowledgements, and usage. Hidden private chain-of-thought is outside the evidence pack.

The next analysis should reconstruct packing and classification trajectories at each accepted student segment and correction. For packing, measure time spent after a verified grasp, transfer distance per correction, and whether takeover follows an actual execution failure. For classification, track which objects receive credit before timeout and whether repeated corrections revisit the same state. These are testable hypotheses from the current evidence.

After any prompt, authority, or cadence change, freeze a new method identity. Rerun a multi-layout exact-case policy, hybrid, and direct-control panel with matched hardware and provider routes. Record valid coverage, native score, paired wins, correction ratio, decision latency, token volume, GPU-hours, and excluded attempts. That design would distinguish quality improvements from extra compute and allow a specific harness change to be assessed.

7. Conclusion

starRoboHarness records the path from policy proposal to reasoner decision, guarded robot action, and native outcome. Historical paired data show gains on some tasks, no success improvement on classification, and a large packing score regression. Reasoning overhead is substantial. The case table and source-digest checks make these observations auditable. The next experiment must test the failure mechanisms under matched resources.

References

- [1] StarVLA Community. *StarVLA: A Lego-like Codebase for Vision-Language-Action Model Developing*. arXiv:2604.05014, 2026.
- [2] GPT-as-Policy contributors. *GPT as Policy*. <https://github.com/anonymous-report-421/GPT-as-Policy>, 2026.
- [3] M. Jia et al. *Agent as Policy for Robotic Manipulation*. arXiv:2609.12541, 2026.
- [4] T. Chen et al. *RoboDojo: A Unified Sim-and-Real Benchmark for Comprehensive Evaluation of Generalist Robot Manipulation Policies*. arXiv:2607.04434, 2026.

A. Ten-task descriptive hybrid index

Block	Task	Hybrid succ.	Hybrid score
Original	Organize table	0/5	65.0
Original	Classify by language	0/5	0.0
Original	Imitate sorting	1/5	36.0
Original	Arrange largest number	1/5	37.0
Original	Fold clothes	4/5	80.0
Added	Put bottles in bin	4/5	82.0
Added	Classify objects	0/5	22.0
Added	Build tower	4/5	86.0
Added	Make Kong	3/5	60.0
Added	Pack objects in box	0/5	2.0
Total	50 cases	17/50	47.0

Table 4 | All 50 selected first-generation hybrid terminal cases, grouped by historical block. Success is shown as completed cases over valid cases. This descriptive index is not a separate experiment.

B. Supplementary development cases

Dev task	Native result	Score	Steps	Policy calls	Corrections
Classify by language	Timeout	10	1,100	146	481
Arrange largest number	Success	100	487	34	0
Make Kong	Success	100	381	27	0
Total / mean	2/3 success	70	1,968	207	481

Table 5 | Later dev validation: one standard g0/10 layout per task. Score in the final row is the three-case mean. Other final-row quantities are totals.

The later dev method completed three standard g0/10 episodes. Language classification timed out at 1,100 control steps with score 10. Arrange largest number succeeded at 487 steps with score 100, and Make Kong succeeded at 381 steps with score 100. The three-case mean score is 70 and two cases succeed. No pure-policy comparator was rerun at the same time. The receipts record 30.91 million input tokens, 29.83 million cached input, and 72.6 thousand output tokens as usage counters. The classification episode accounts for all 481 correction steps in this small run. The two successful episodes record zero correction steps. This pattern identifies classification as the immediate trajectory-analysis target and cautions against crediting corrections for the two successes.

Off-panel build_tower method	Native result	Score	Steps	Policy calls
QwenPI_v3, 16-step	Timeout	10	1,050	66
QwenPI_v3, fixed-15 ablation	Timeout	0	1,050	70
Astra + QwenPI_v3	Success	100	761	52
Astra Direct	Timeout	0	1,050	0

Table 6 | Archived alpha development evidence for one build_tower layout. Astra Direct issued 210 direct decisions. Its zero policy-call count reflects the absence of a student policy. No row belongs to the frozen panels.

The archived first prototype also has one off-panel build_tower case under three methods. Pure QwenPI_v3 timed out with score 10. Astra plus QwenPI_v3 succeeded with score 100 and zero correction steps. Astra Direct timed out with score 0 after 1,050 control steps and about 132 minutes. Table 6 gives the receipts, including the fixed-15 policy ablation. This single layout confirms that all execution paths reached native termination. It cannot estimate population performance or explain the later build-tower panel.

C. Historical experiment provenance

Stage	Evidence boundary	Result and interpretation
v1	17–18 Sep alpha snapshot	One off-panel build__tower case per main method. Hybrid native success, with pure policy and Astra Direct timeouts. The formal 75-episode target had not started.
main	21–25 Sep frozen campaigns	Original five-task 25-pair result, followed by five added hybrid tasks and later 25 exact pure-policy counterparts. Historical ten-task hybrid summary is 17/50.
dev	25 Sep source freeze aa14b9d	Three single-layout native hybrid outcomes (2 successes, 1 timeout), followed by recovery-lineage change 504e0e2. Development validation, not held-out comparison.

Table 7 | Versioned evidence lineage. Stage names refer to development history, not randomized experimental treatments. Current branch pointers must not be used to infer historical method identity.

The first prototype established model-neutral contracts, persistent reasoner tools, frozen case identities, runtime receipts, and failure-aware monitoring. Subsequent main work added receipt-based recovery, same-node baseline collection, isolation of concurrent reasoner sessions, and a public package boundary. The later dev work introduced episode-local agent workspaces, task-context ledgers, provider-failure classification, and guarded continuation for independent cases. Several of these changed between historical observations. The available data do not isolate the effect of a branch change.